Innopolis University

09.03.01 · Computer Science

Bachelor's Thesis Defense

Bridging the Enforcement Verification Gap

Detecting Divergence Between Declared and Actual Security Posture in Kubernetes Containerized Environments

AUTHOR

Yehia Sobeh

SUPERVISOR

Kirill Saltanov

Innopolis · 2026

Motivation: The Problem with Declarative Security

Why This Matters

Over **60% of organizations** running containers in production use Kubernetes *(CNCF survey, 2024)*.

This creates a large attack surface:

hundreds of containers per cluster
third-party images of varying trustworthiness
continuous scaling and dynamic workloads

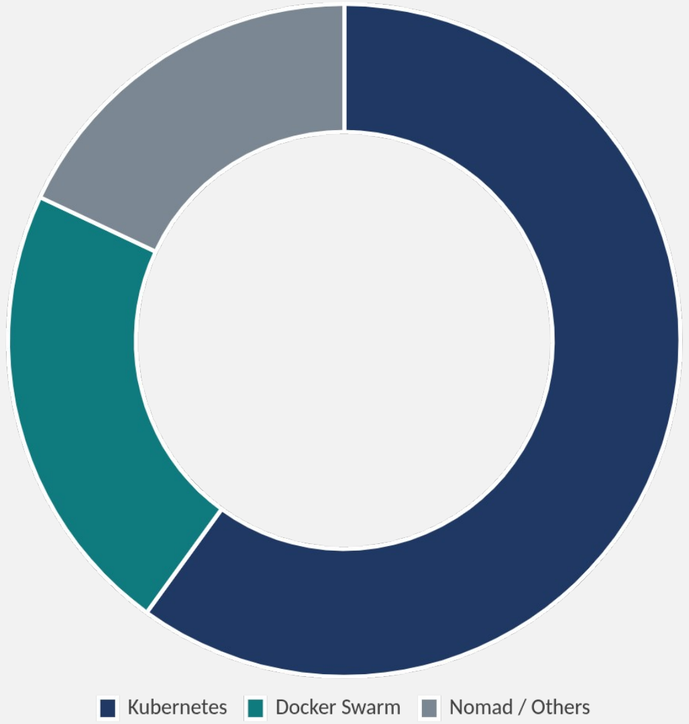
All of this is secured by **declarative YAML policies**.

Critical Assumption

Every existing tool (OPA Gatekeeper, Kyverno, Kubescape, Trivy) assumes that **if the Kubernetes API reports a security property, the container runtime has faithfully enforced it** .

This thesis demonstrates: that assumption does not always hold. A privileged container that declares seccomp filtering has that filter silently removed by the Docker runtime — and no existing tool detects this.

Container Orchestrator Share (Production)



CNCF Annual Survey, 2024 · production deployments

Why this matters at scale

A silent enforcement failure in a single Docker default propagates to **millions of production pods** across the industry.

The Enforcement Verification Gap

Chapter 1 · Problem Statement

From YAML declaration to kernel enforcement, the pod spec passes through **four translation hops** — each one can silently drop a restriction.

HOP 1

Kubernetes API

Pod securityContext declared by the operator.

HOP 2

Kubelet → CRI

Translates the spec into a Container Runtime Interface request.

HOP 3

Container Runtime

Docker / containerd / CRI-O generates the OCI runtime spec.

HOP 4

runc + Linux Kernel

Configures namespaces, cgroups, capabilities, seccomp.

Where Existing Tools Look — and Where the Gap Lives

	Hop 1	Hop 2	Hop 3	Hop 4 (kernel)
OPA / Kyverno	✓	—	—	—
Falco / KubeArmor	—	—	—	behaviour only
k8s-psa (this work)	✓	—	✓	✓

Formal Definition

The **enforcement verification gap** is the absence of tooling that systematically verifies whether the security context declared in a Kubernetes pod specification is faithfully enforced at the container runtime level.

Research Questions

RQ1. Does kernel-level state actually diverge from the declared spec, and under what conditions?

RQ2. Can a lightweight node-resident agent detect these divergences with bounded overhead?

RQ3. What categories of divergence occur in practice, and how should each be classified?

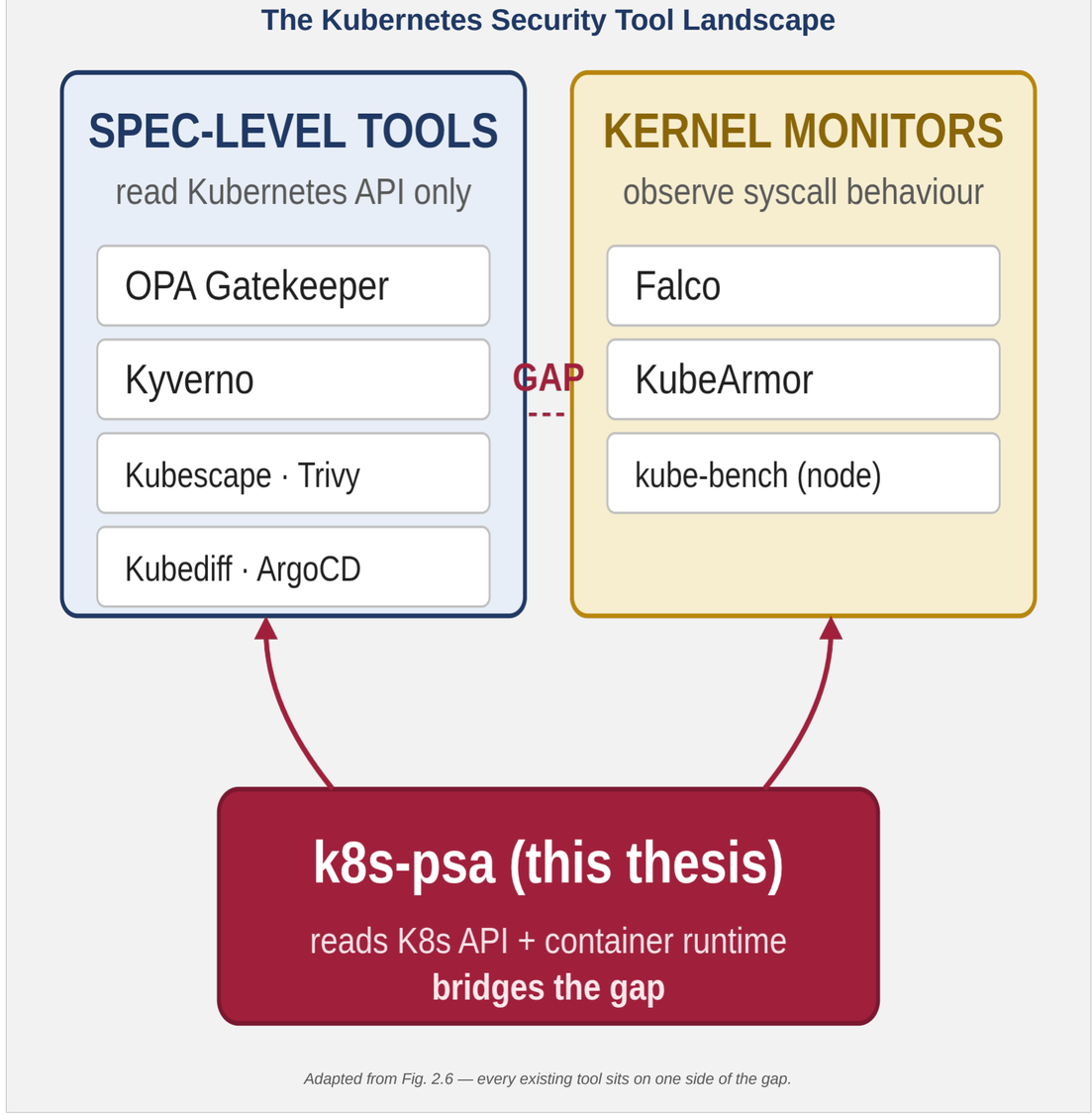
Literature Review: No Existing Tool Bridges Both Ends

Chapter 2 · Literature Review

A systematic walkthrough of every tool category in the Kubernetes security ecosystem confirms that the gap is real and unaddressed.

Tool / Category	Data source	Drift?
OPA Gatekeeper	K8s API	No
Kyverno	K8s API	No
Kubescape / Trivy Operator	K8s API	No
Falco / KubeArmor	Kernel (eBPF, syscalls)	No *
Kubediff / ArgoCD / FluxCD	K8s API + Git	No
kube-bench	Node filesystem	No
k8s-psa (this thesis)	**K8s API + container runtime**	Yes ✓

** Falco detects syscall anomalies but cannot tell whether a declared restriction silently failed to apply. Wong et al., Devi Priya et al., and CloudSecBurrito name this gap as an open research direction.*



Adapted from Fig. 2.6 — every existing tool sits on one side of the gap.

Seccomp: The Security Primitive Most Affected

Chapter 2 · Seccomp in Kubernetes

A Linux kernel BPF filter declared in YAML, translated through four layers, and silently lost in the most common privileged-mode case.

Three Seccomp Profile Types

Unconfined

No filter at all. Every syscall is permitted. Historically the default. Exposes `mount`, `ptrace`, `kexec_load`, `reboot`.

RuntimeDefault

The runtime's built-in BPF allowlist. Blocks ~36 syscalls historically used in container escapes while permitting all common workload calls.

Localhost

A custom BPF profile loaded from the node filesystem. Strictest option — allows only what the specific application needs.

Three Properties Make Seccomp Vulnerable to Silent Failure

1. Asymmetric Default

Seccomp is opt-in. A pod without a `seccompProfile` runs **Unconfined** unless the kubelet is explicitly configured otherwise. KEP-2413 started changing this; migration is incomplete.

2. Privileged-Mode Override

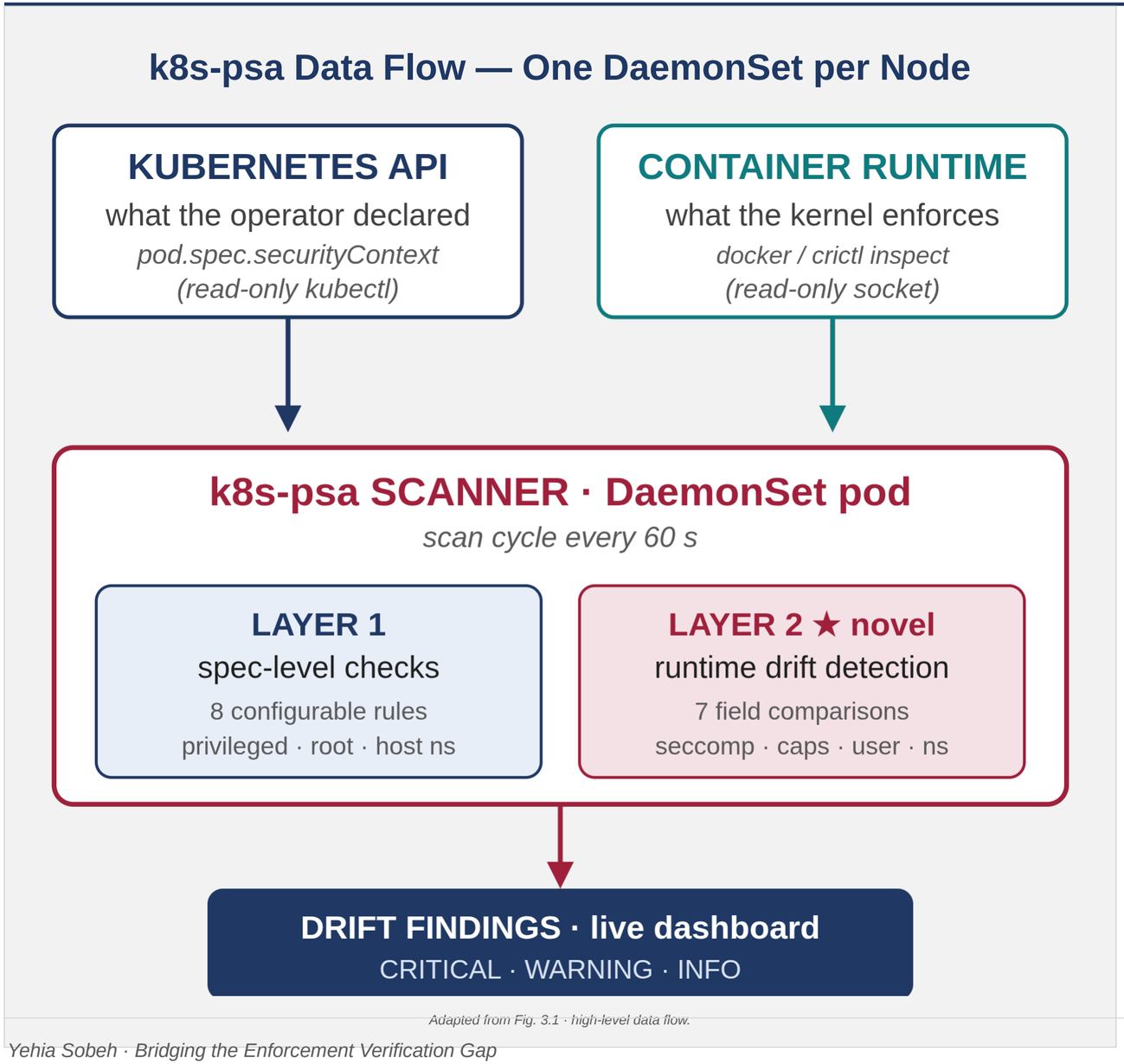
Docker silently **strips** declared seccomp profiles from privileged containers (Moby issue #41321). The spec still reports `RuntimeDefault` — the kernel applies no filter. *This is the central enforcement gap.*

3. Opaque Metadata

When `cri-dockerd` forwards a `RuntimeDefault` declaration, no explicit `SecurityOpt` entry is recorded. `docker inspect` cannot distinguish a protected container from an unprotected one.

k8s-psa: Architecture Overview

Chapter 3 · Design & Methodology



Adapted from Fig. 3.1 · high-level data flow.

Design Principle

Verification requires **two independent data sources** : the K8s API for declared intent, the runtime for kernel-enforced reality.

Layer 1 — Specification

API only. Functionally equivalent to OPA / Kyverno.

privileged · root · host ns · seccomp · caps · read-only root

Layer 2 — Drift Detection ★ novel

API + runtime. Spec field vs. live runtime field.

privileged · user · capabilities · **seccomp** · rootfs+noNewPriv · host ns · masked paths

Severity: CRITICAL · WARNING · INFO

Two Design Decisions That Make Layer 2 Work

Chapter 3 · Design Details

1. Capability Normalization

The Linux kernel grants **14 default capabilities** to every non-privileged container, even when the spec lists zero. A naïve comparison emits 14 false positives per pod.

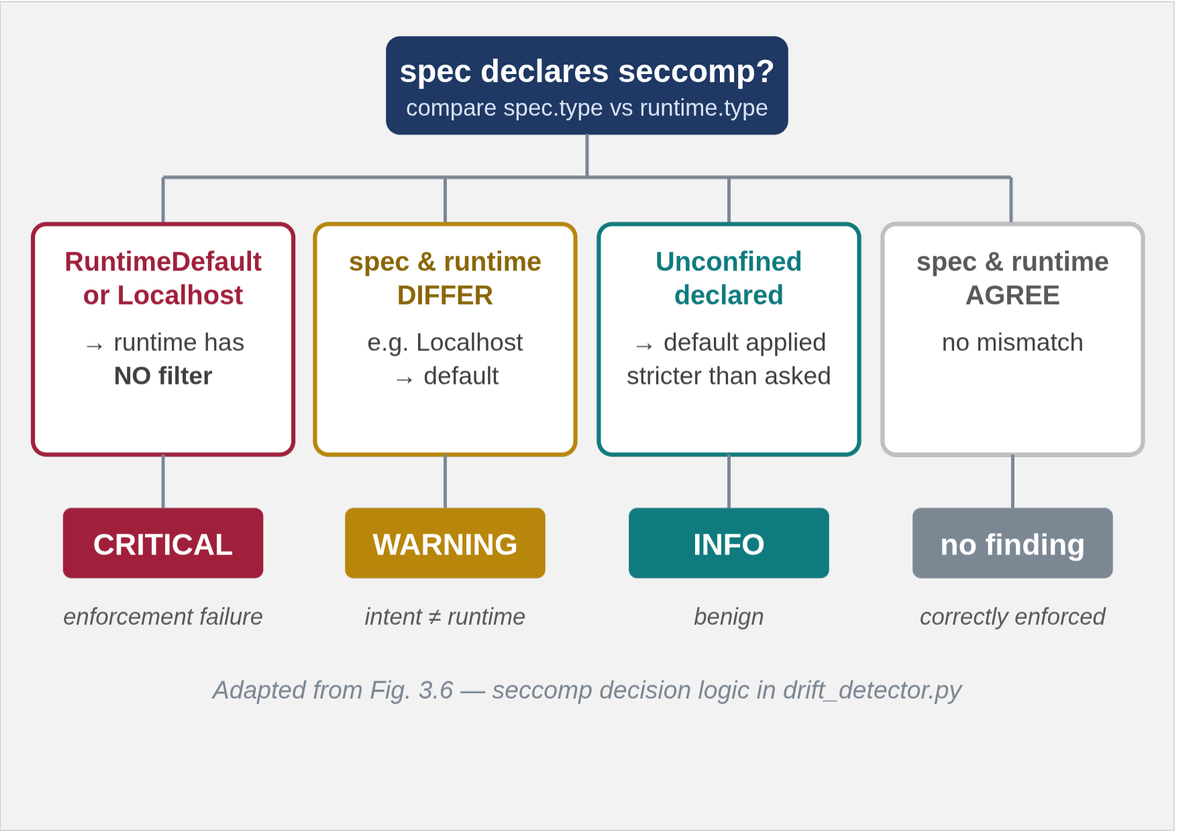
```
def compute_expected(add, drop):
    if "ALL" in drop:
        expected = set()
    else:
        expected = DEFAULTS - set(drop)
    expected = expected | set(add)
    return expected
```

For privileged containers

Docker grants all 41 capabilities. The algorithm **deliberately does NOT** expand the expected set to 41. Result: 27 **WARNING** findings — correctly surfacing the inflation that privileged mode silently introduces.

2. Seccomp Comparison Logic

Four rules govern when a seccomp mismatch is a finding:



Adapted from Fig. 3.6 — seccomp decision logic in drift_detector.py

Seven Purpose-Built Test Pods

Chapter 3 · Evaluation Design

Designed to isolate individual variables — the scientific control at the heart of the evaluation.

#	Pod Name	Security Configuration	Scientific Role
1	Fully Hardened	drop ALL · runAsUser 1000 · RuntimeDefault · readOnlyRootFs · no escalation	Negative control — expect zero findings
2	No Security Context	No `securityContext` at all	Empty baseline
3	**Privileged + Seccomp ★**	`privileged: true` + `seccompProfile: RuntimeDefault`	**KEY scenario — exposes the gap**
4	Privileged, No Seccomp	`privileged: true` (no seccomp declaration)	Scientific control for Pod 3
5	Partial Security	drop NET_RAW, SYS_ADMIN + RuntimeDefault + runAsUser 1000	Tests partial capability drops
6	No Privilege Escalation	`allowPrivilegeEscalation: false` + full hardening	Tests noNewPrivileges mapping
7	Host Namespaces	hostPID + hostNetwork + hostIPC	Tests namespace mode translation

Controlled Experiment — Pod 3 vs. Pod 4

Both pods declare `privileged: true` and end up with **identical runtime state** — all 41 capabilities, no seccomp, no masked paths. The only variable that differs is the seccomp declaration in Pod 3's spec. **If the tool is correct:** Pod 3 → 1 CRITICAL · Pod 4 → 0 CRITICAL. That asymmetry proves specificity.

Independent confirmation: the same scenario was tested against four real Kubernetes control-plane components — *etcd, kube-apiserver, kube-controller-manager, kube-scheduler*.

Implementation: Python 3.12 · 9 Modules · 2 Runtime Adapters

Module Organization

Data Extraction

`pod_scanner.py`

`spec_extractor.py`

`runtime_inspector.py`

Calls kubectl, parses pod JSON, abstracts the container-runtime backend.

Analysis

`policy_engine.py`

`normalizer.py`

`drift_detector.py`

Layer 1 policy checks, capability normalization, Layer 2 field-by-field comparisons.

Output

`alert_manager.py`

`main.py`

`dashboard/server.py`

NDJSON alerts on host volume, scan-loop orchestrator, real-time Chart.js dashboard.

Runtime Adapters

Docker Adapter

Calls `docker inspect` and parses `HostConfig.SecurityOpt`, `CapAdd/Drop`, `Privileged`, `ReadonlyRootfs`, `PidMode`, `NetworkMode`, `IpcMode`, `Config.User`.

containerd Adapter

Calls `crictl inspect` and reads `info.runtimeSpec` (full OCI spec) directly — full seccomp BPF rule set from `linux.seccomp`. **Eliminates the observability gap.**

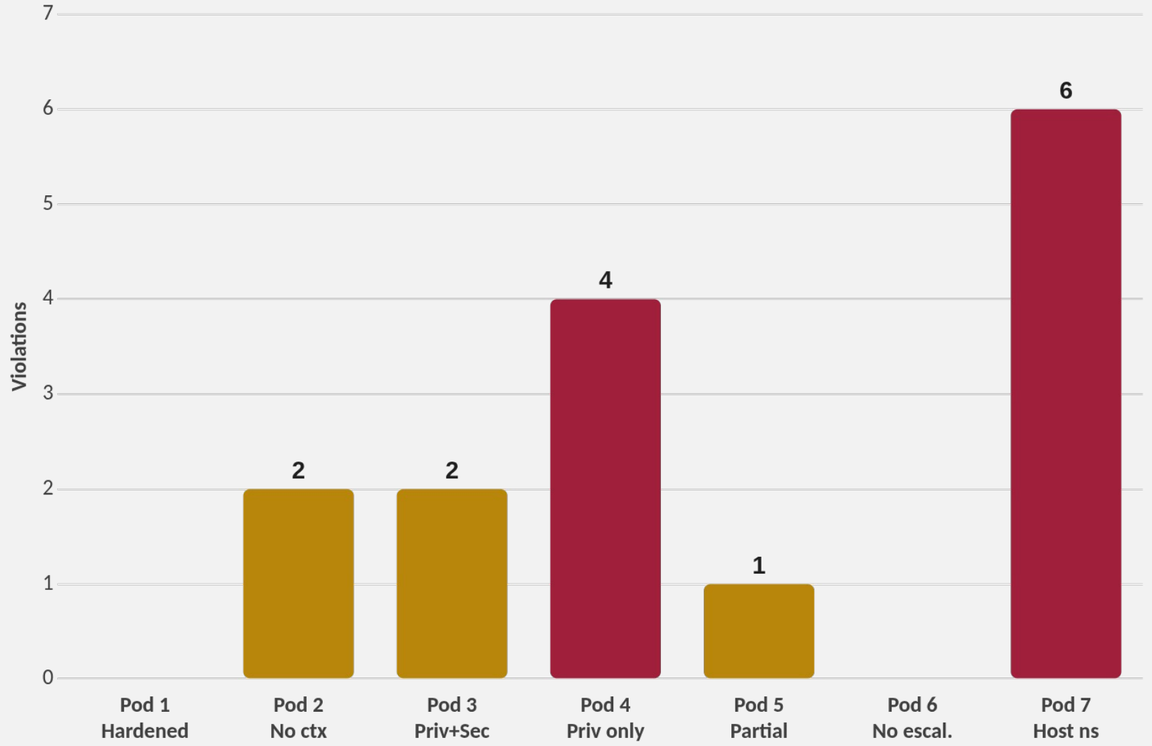
44 unit tests · runs in **0.2 s** · no cluster needed | One-command deploy: `kubectl apply -f deployments/all-in-one.yaml`

Layer 1 Results: Specification-Level Violations

Chapter 5 · Evaluation — Layer 1

Eight policy checks across seven test pods — baseline correctness confirmed before interpreting Layer 2 findings.

Pod	Test Pod	Violations
1	Fully Hardened	0
2	No Security Context	2
3	Privileged + Seccomp	2
4	Privileged, No Seccomp	4
5	Partial Security	1
6	No Privilege Escalation	0
7	Host Namespaces	6



Finding 1: Docker Silently Strips Seccomp (TRUE POSITIVE)

Chapter 5 · Central Result — Pod 3 vs. Pod 4 Controlled Comparison

Pod 3 — privileged: true + seccomp: RuntimeDefault

Kubernetes Spec:

```
privileged: true
seccompProfile:
  type: RuntimeDefault
```

Container Runtime (what Docker does):

```
capabilities: all 41 granted seccomp:      STRIPPED
```

→ **1 CRITICAL** + **27 WARNING**

Pod 4 — privileged: true (no seccomp declared) — CONTROL

Kubernetes Spec:

```
privileged: true # no seccompProfile
```

Container Runtime (what Docker does):

```
capabilities: all 41 granted
seccomp:      stripped
```

→ **0 seccomp findings** + **27 WARNING**

Both pods have **byte-identical runtime state**. The only difference is one line of YAML — **1 CRITICAL** for Pod 3 vs. **0** for Pod 4. Controlled proof of the enforcement gap.

Independent confirmation — Kubernetes control plane (Table 5.3)

System Pod	CRIT	WARN	INFO
etcd-minikube	1	0	1
kube-apiserver-minikube	1	0	1
kube-controller-manager-minikube	1	0	1
kube-scheduler-minikube	1	0	1
kube-proxy	0	27	1
storage-provisioner	0	0	1

Why this matters

The four privileged control-plane pods reproduce Pod 3's finding on a real Kubernetes deployment. Invisible to **OPA Gatekeeper, Kyverno, Kubescape, Trivy Operator** .

Documented in **Moby #41321** since 2020 — never previously detected by any verification tool.

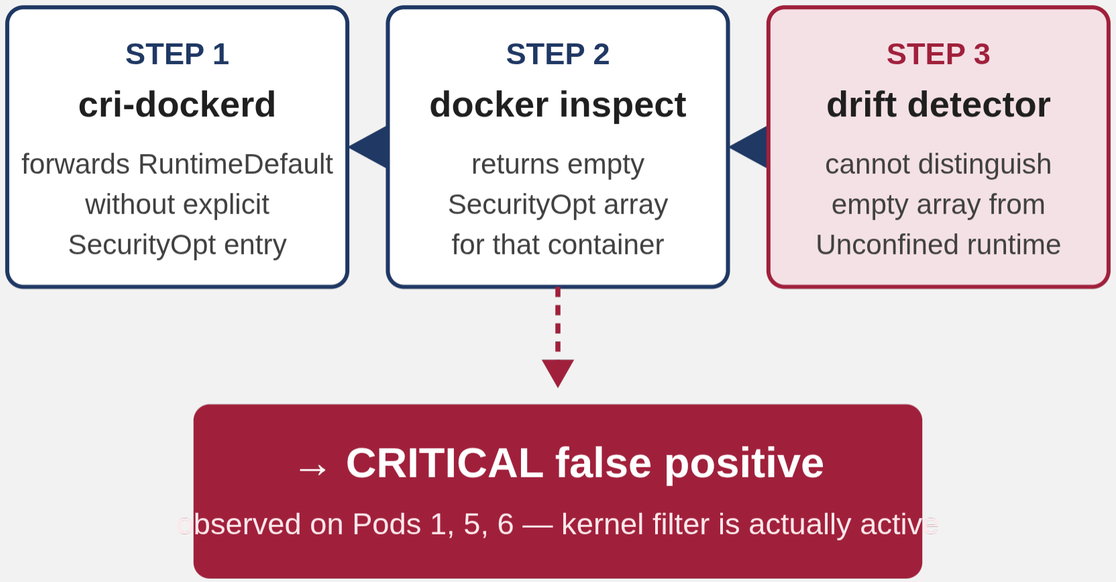
Finding 2: docker inspect Hides Implicit Seccomp Defaults

Chapter 5 · Observability Artefact — distinct from Finding 1

What I Observed

Pods **1, 5, and 6** — non-privileged, correctly declaring `RuntimeDefault` — also produced CRITICAL seccomp findings. **But these containers are actually protected.** The kernel is applying a seccomp filter. The findings are false positives.

Root Cause — Three-Step Chain



Distinct from Finding 1 — here the protection IS present, only visibility is missing.

Lesson

Runtime metadata is a **lossy abstraction**. Any tool that relies on `docker inspect` for seccomp verification will produce these false positives.

Docker acknowledges this in **CIS Docker Benchmark check 5.21** — an empty `SecurityOpt` means the default profile is active, not that seccomp is disabled.

Definitive Fix — Read the Kernel

```
# Bypass docker inspect entirely $ cat /proc/$PID/status | grep Seccomp
Seccomp: 2
# 2 = BPF filter mode active
```

Finding 1 (real risk)	Finding 2 (artefact)
Privileged containers — enforcement gap. The kernel literally has no filter.	**Non-privileged containers — observability gap.** The filter is active, but metadata hides it.

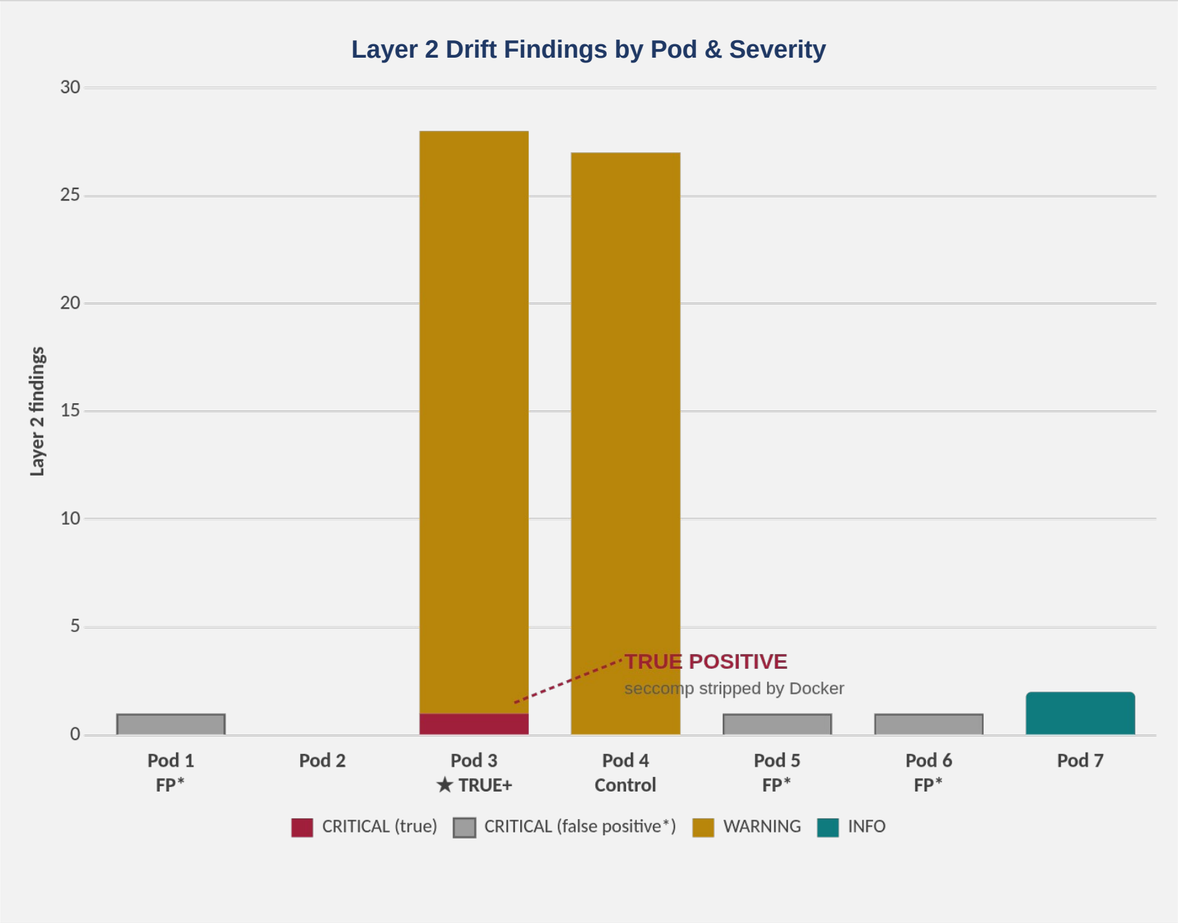
Full Evaluation Results — Layer 2 Drift Findings

Chapter 5 · k8s-psa Live Dashboard

7 + 6
pods evaluated | test + system

3
scan cycles · 60 s each

8
unique CRITICAL · 4 test + 4 system



1
CRITICAL · Pod 3 ✓
privileged + seccomp declared

0
CRITICAL · Pod 4 ✓ control
privileged · no seccomp declaration

27
WARNING per privileged pod
41 granted − 14 default = capability inflation

Every predicted finding appeared. The Pod 3 vs Pod 4 asymmetry proves the tool detects the enforcement gap, not general privileged-mode noise.

Solid bar = true positive · Grey bar = observability false positive (cri-dockerd metadata gap · Finding 2)

Answering the Three Research Questions

Chapter 5 · Discussion

#	Question	Answer	Status
RQ1 Empirical	**Does divergence actually exist?**	Yes. Docker silently strips declared seccomp from privileged containers. The K8s API reports `RuntimeDefault`; the kernel applies no filter. Confirmed by Pod 3 vs. Pod 4 controlled comparison.	CONFIRMED
RQ2 Engineering	**Can a lightweight tool detect it?**	Yes. k8s-psa successfully detects enforcement divergences with a lightweight footprint — completing full scan cycles in seconds, consuming minimal memory and CPU, and requiring no modifications to existing cluster security mechanisms.	CONFIRMED
RQ3 Taxonomy	**What categories of divergence occur?**	Three: (1) **enforcement gap** — seccomp stripped from privileged containers; (2) **capability inflation** — 27 extra caps granted by privileged mode; (3) **observability artefact** — Docker metadata hides implicit defaults.	3 CATEGORIES

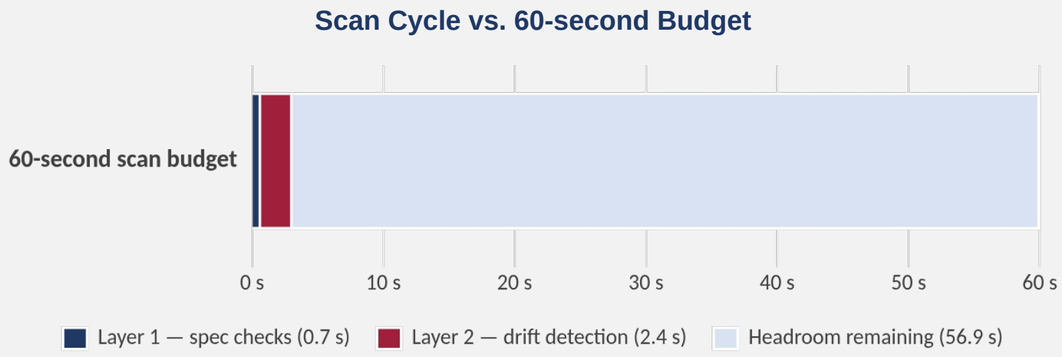
Performance Footprint

≈ 3.1 s
Scan cycle | 16 pods / 18 containers

64 + 32 MiB
Memory | scanner + dashboard

< 100 m
CPU usage | against 100 m limit

44 · 0.2 s
Unit tests | no cluster required



5.2% of the 60 s budget used · 56.9 s headroom (Table 5.5).

Limitations — Honest Scope

Chapter 6 · Limitations

Four scope decisions bound the conclusions of this work. None of them invalidate the central finding — each maps cleanly to a future-work item on the next slide.

1 · Single Runtime Tested

Evaluation uses Minikube + Docker only. The containerd adapter is implemented and unit-tested, but not exercised in cluster experiments.

Impact: the seccomp-stripping finding is confirmed for Docker; cross-runtime reproducibility is open.

2 · Metadata-Based Seccomp Check

Layer 2 reads `docker inspect` rather than the kernel directly. This is precisely what produced the false positives on Pods 1, 5, and 6 — Finding 2.

Impact: the observability gap is a known, characterised limitation, not an unknown unknown.

3 · Single-Node Cluster

Multi-node clusters, NetworkPolicy enforcement, and service-mesh interactions (Istio, Linkerd) were not in scope.

Impact: findings are conservative — they hold per-node; multi-node behaviour is open.

4 · CLI-Based Inspection

The tool shells out to `kubectl` and `docker` rather than using native Python client libraries.

Impact: some parsing fragility under version skew; functionally complete on the tested versions.

Future Work — Natural Next Steps

Chapter 6 · Future Work

Each limitation maps directly to a concrete next step. The two findings are mature enough that publication is the recommended next move.

1 · Kernel-Level Seccomp Verification

Read `/proc/<pid>/status` directly. A value of `Seccomp: 2` confirms BPF mode regardless of what Docker's metadata reports.

Eliminates the observability gap entirely — addresses Limitation 2.

2 · Cross-Runtime Evaluation

Deploy on containerd (kind, GKE, EKS) and CRI-O (OpenShift). Does the seccomp-stripping reproduce? Is the observability gap absent on containerd?

Addresses Limitation 1.

3 · Mutating-Webhook Drift

RESEARCH

Admission webhooks (PSA, custom controllers) can silently add or modify seccomp profiles *after* the user submits the spec. A potential third class of enforcement gap.

New research direction — multi-node and webhook-aware.

4 · Publication

RECOMMENDED

The two findings — the true enforcement gap and the observability artefact — together form a novel, reproducible, and well-evidenced contribution suitable for publication at a security venue.

Strong empirical signal · controlled experiment · open-source artefact.

Conclusion: Four Contributions

Chapter 6 · Summary

1. A New Verification Tool

k8s-psa — first open-source tool to systematically compare Kubernetes declarations against actual container runtime state across 7 security dimensions.

2. True Enforcement Gap

Empirical evidence that Docker strips declared seccomp from privileged containers — a vulnerability invisible to every existing specification-only tool.

3. Observability Gap

`docker inspect` omits implicit seccomp defaults — any security tool relying on this metadata inherits the false-positive problem.

4. Open-Source Eval Framework

44 unit tests · 7 deployment-level test cases · one-command deploy · real-time Chart.js dashboard for continuous monitoring.

Thank You

Questions welcome.

Yehia Sobeh · Supervisor: Kirill Saltanov · Innopolis · 2026